

C03 ASSESSMENT TOOL 1(Coding Challenge)

Name: Y.Thanushma

Registration Number: 192425355

Course Code: MLA0304

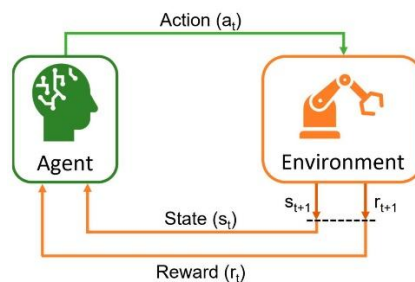
1.Reinforcement Learning (RL) Framework

Introduction

Reinforcement Learning is a machine learning method where an agent learns by interacting with an environment. It takes actions, receives rewards or penalties, and improves its decisions to maximize future rewards.

Basic interaction:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ New State



Agent–Environment Interaction

At each step:

1. The agent observes the state.
2. It selects an action.
3. The environment responds with a reward.
4. The agent moves to a new state.
5. The process repeats.

Importance of State, Action and Reward

- State: Represents the current situation of the environment.

- Action: The decision made by the agent.
- Reward: Feedback indicating whether the action was good or bad.

For example, in robot navigation:

Component	Example
State	Robot position and obstacles
Action	Move left/right/forward
Reward	+10 for reaching target, -10 for hitting obstacle

Cumulative Reward

The agent aims to maximize the total future reward: $G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$

where γ is the discount factor.

For example, if rewards are 5, 4 and 10 with $\gamma = 0.9$:

$$G = 5 + (0.9)(4) + (0.9)^2(10)$$

$$G = 16.7$$

Thus, the cumulative reward is 16.7.

Real-World Example

In a self-driving car:

- State: Speed, traffic and road position
- Action: Accelerate, brake or turn
- Reward: Positive for safe driving and negative for collisions

The car may choose braking even though it reduces speed because it prevents a collision and gives a better long-term reward.

Conclusion

RL enables intelligent decision-making through states, actions and rewards. Cumulative reward helps the agent consider future consequences, allowing it to learn better decisions rather than focusing only on immediate rewards.

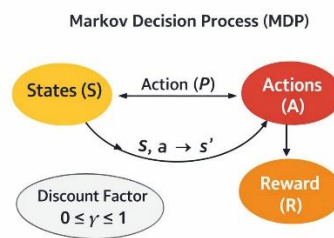
2. Markov Decision Process (MDP)

A Markov Decision Process (MDP) is a mathematical framework used to model decision-making problems where an agent interacts with an environment.

An MDP is represented as:

Where:

- State Space (S): All possible situations of the agent.
- Action Space (A): All possible actions the agent can take.
- Transition Probability (P): Probability of moving from one state to another after an action.
- Reward Function (R): Gives a numerical reward or penalty for an action.
- Discount Factor (γ): Determines the importance of future rewards



Example

For a robot navigation system:

- State: Robot's current position
- Actions: Move left, right, forward
- Transition: Probability of reaching the intended position
- Reward: +10 for reaching the target, -10 for hitting an obstacle
- Discount factor: 0.9 to consider future rewards

Thus, MDP provides a structured way to represent and solve sequential decision-making problems.

3. Policy and Value Functions in Reinforcement Learning

1. Introduction

In Reinforcement Learning, the policy tells the agent which action to choose, while value functions help evaluate the long-term benefit of states and actions.

2. Policy

A policy () is a strategy that defines which action the agent should take in a particular state.

$$\pi(a|s)=P(A=a|S=s)$$

For example, a robot may choose **move forward** when the target is in front of it.

3. Value Functions

Value functions estimate the **long-term rewards** that an agent can obtain.

State-ValueFunction:

Measures the expected future reward from a particular state. $V\pi(s)=E[R_t+1+\gamma V\pi(S_{t+1})]$

Action-ValueFunction:

Measures the expected future reward for taking action in state .

$$Q\pi(s,a)=E[R_t+1+\gamma Q\pi(S_{t+1},A_{t+1})]$$

4. Bellman Equation

The Bellman Equation expresses the value of a state as: $V(s)=R+\gamma V(s')$

It means:

Current Reward + Discounted Future Value = Total Value

For example, if:

- Reward = 5
- Future value = 10
-

Then: $V(s)=5+(0.9)(10)=14$

So, the long-term value is **14**.

5. Role in Decision-Making

The agent compares the values of different actions and chooses the action with the highest expected long-term benefit.

For example:

- Action A $\rightarrow$ Value = 8
- Action B $\rightarrow$ Value = 14

The agent chooses **Action B** because it provides a higher expected long-term reward.

Conclusion

The policy determines which action to take, while state-value and action-value functions evaluate the long-term benefits of states and actions. The Bellman Equation connects immediate rewards with future rewards and helps the RL agent make optimal decisions.

4. Exploration–Exploitation Trade-off in RL

Introduction

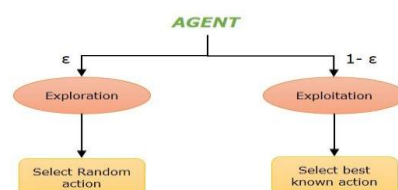
The exploration–exploitation trade-off is an important problem in Reinforcement Learning. The agent must decide whether to explore new actions or exploit the best-known action.

Exploration and Exploitation

- **Exploration:** Trying new actions to discover better rewards.
- **Exploitation:** Choosing the action that currently has the highest expected reward.

Too much exploration may waste time, while too much exploitation may prevent the agent from discovering better actions.

Epsilon Greedy Action Selection



ϵ -Greedy Strategy

In ϵ -greedy, the agent:

- Explores with probability ϵ .
- Exploits the best action with probability $1-\epsilon$.

For example, if: $\epsilon=0.1$

the agent explores 10% of the time and exploits 90% of the time.

Softmax Strategy

The Softmax approach selects actions according to their estimated values. Actions with higher values have a higher probability of being selected.

$$P(a) = \frac{e^{Q(a)}}{\sum e^{Q(a)}}$$

Here, T is the temperature parameter.

Comparison

Strategy	Main Idea	Advantage
ϵ -Greedy	Random exploration with probability ϵ	Simple and easy to implement
Softmax	Selects actions based on their values	More balanced exploration

Conclusion

Both strategies balance exploration and exploitation. ϵ -greedy is simple and efficient, while Softmax provides more controlled exploration based on action values.

5. Reward Shaping in Reinforcement Learning

Introduction

Reward shaping is the process of providing additional rewards or penalties to guide an RL agent toward desirable behavior. It can make learning faster and more efficient.

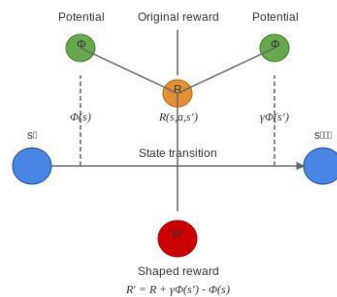
How Reward Shaping Works

Instead of giving a reward only when the final goal is achieved, smaller rewards can be provided for useful intermediate actions.

For example, in robot navigation:

- Reaching target $\rightarrow +10$
- Moving closer to target $\rightarrow +2$
- Moving away $\rightarrow -1$
- Hitting obstacle $\rightarrow -10$

This gives the agent useful feedback during learning.



Impact on Learning

Reward shaping can:

- Speed up learning.
- Guide the agent toward the correct behavior.
- Reduce unnecessary actions.
- Help avoid obstacles and poor decisions.
- Prevent the agent from repeatedly choosing harmful actions.

Example

Without reward shaping, a robot may receive +10 only after reaching the destination.

With reward shaping, it receives small rewards while moving closer to the destination. Therefore, the robot learns the correct path faster.

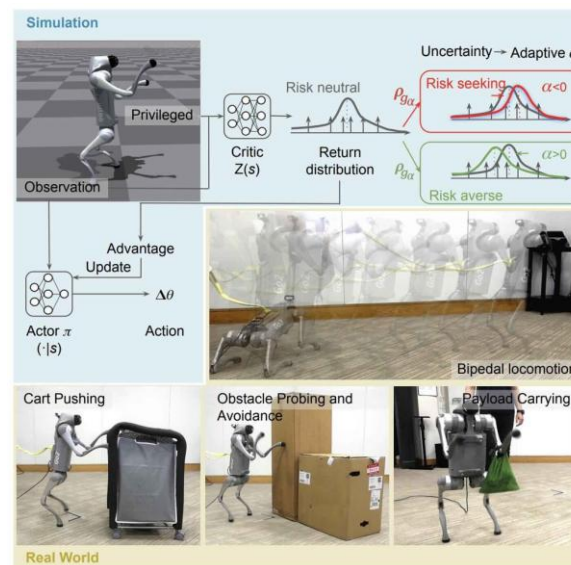
Conclusion

Reward shaping provides useful intermediate feedback and accelerates RL learning. Properly designed rewards help the agent avoid undesirable behavior and achieve the final goal more efficiently.

6. Challenges of Applying RL in Real-World Robotics

Introduction

Applying Reinforcement Learning to real-world robots is challenging because robots operate in uncertain, changing, and sometimes dangerous environments.



2. Major Challenges

- **Safety:** Wrong actions may damage the robot or harm people. Safe exploration is therefore necessary.
- **Sample Efficiency:** RL may require a large number of trials to learn a good policy. Real-world trials are costly and time-consuming.
- **Environment Variability:** Lighting, obstacles, noise, and object positions can change, reducing performance.

3. Impact on Performance

A robot trained in one environment may perform poorly in another because the conditions are different. Therefore, RL systems need robust policies and safe learning methods.

Conclusion

Safety, sample efficiency, and environment variability are major challenges. Improving these factors makes RL more reliable and practical for real-world robotics.

7. Influence of Discount Factor (γ)

Introduction

The discount factor (γ) determines how much importance an RL agent gives to future rewards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3}$$

Effect of γ

- **Low γ (near 0):** Agent focuses mainly on immediate rewards.
- **High γ (near 1):** Agent considers future rewards more strongly.

For example, if: $R_1=5, R_2=10$

with $\gamma=0.5$: $G=5+(0.5)(10)=10$

With $\gamma=0.9$: $G=5+(0.9)(10)=14$

Thus, higher γ gives greater importance to future rewards.

Role in Episodic Tasks

In episodic tasks such as games or robot navigation, a suitable γ helps the agent balance immediate actions with the final goal.

Conclusion

A low γ supports short-term decision-making, while a high γ supports long-term planning. Choosing an appropriate γ is important for good performance.

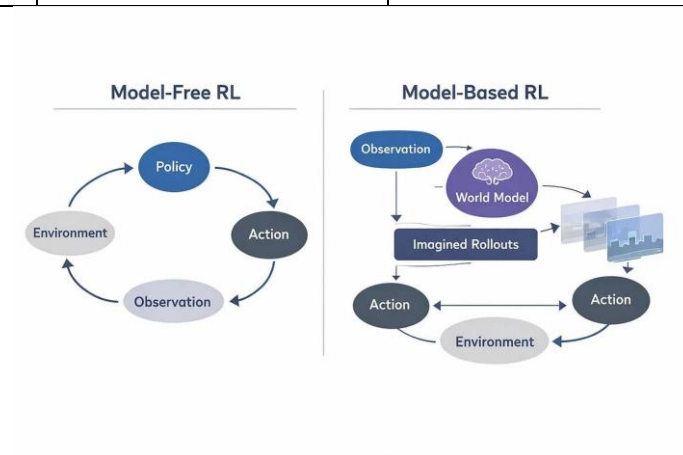
8. Model-Free vs Model-Based RL

Introduction

RL can be divided into model-free and model-based approaches depending on whether the agent uses a model of the environment.

Comparison

Feature	Model-Free RL	Model-Based RL
Environment model	Not required	Required/learned
Learning	Directly from experience	Uses model to predict outcomes
Sample efficiency	Usually lower	Usually higher
Planning	Limited	Strong
Complexity	Simpler	More complex



Strengths and Limitations

Model-Free:

- Simple to implement.
- Works well in complex environments.
- Requires many interactions.

Model-Based:

- Can plan future actions.

- More sample efficient.
- Performance depends on model accuracy.

Conclusion

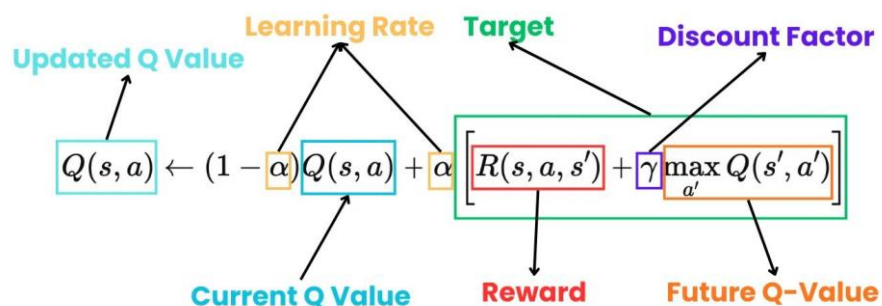
In dynamic environments, model-based RL can adapt efficiently when the environment model is accurate, while model-free RL is simpler and useful when modeling the environment is difficult.

9. Q-Learning Update

Introduction

Q-learning is an off-policy RL algorithm that learns the value of taking an action in a particular state.

The update equation is: $Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$



2. Example Calculation

Suppose:

- Current $Q(s,a)=5$
- Reward $R=10$
- Learning rate $\alpha=0.5$
- Discount factor $\gamma=0.9$
- Maximum future Q-value = 8

Then:

$$Q(s,a)=5+0.5[10+(0.9)(8)-5]$$

$$=5+0.5[12.2]$$

Thus, the Q-value changes from **5** to **11.1**.

Convergence

During training, Q-values are repeatedly updated using new experiences. As the agent gains sufficient experience, the Q-values become stable and approach their optimal values.

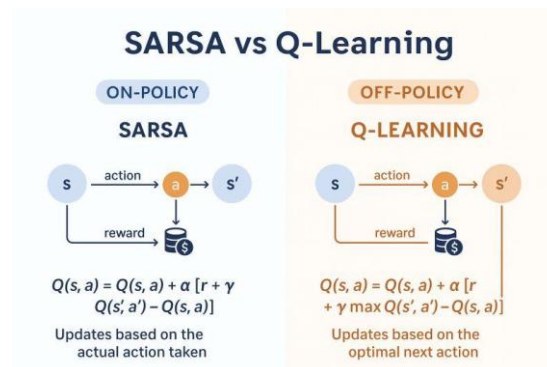
Conclusion

Q-learning improves action selection by repeatedly updating Q-values. With sufficient exploration and training, the values can converge toward optimal action values.

10. On-Policy vs Off-Policy Learning

Introduction

On-policy and off-policy methods differ in how they learn from actions and policies.



On-Policy Learning

An on-policy algorithm learns about the same policy that it follows.

Example: SARSA

It learns from the actions actually selected by the agent.

Off-Policy Learning

An off-policy algorithm learns an optimal policy while following a different behavior policy.

Example: Q-learning

It can learn from experiences generated using exploratory actions.

Comparison

Feature	On-Policy	Off-Policy
Learning policy	Same as behavior policy	Different from behavior policy
Example	SARSA	Q-learning
Exploration	Included directly	Can learn from exploratory data
Advantage	Often safer and stable	More flexible and reusable
Limitation	Less flexible	Can be less stable

Conclusion

On-policy learning is useful when the agent should learn from its actual behavior, while off-policy learning is more flexible because it can learn an optimal policy from different experiences.